Лабораторная работа № 1
«Цветовые модели»

Дисциплина: Компьютерная графика

Выполнил: студент Белов Павел

Группа: 4 Курс: 3

1 Цель работы

Изучить принципы представления цвета в моделях RGB, CMYK и HSV, а также математические основы преобразования цветовых координат между ними.

Разработать настольное приложение на Python, которое позволяет выбирать цвет и интерактивно изменять его компоненты. При любом изменении должны автоматически обновляться числовые значения во всех трёх моделях и визуальный образец выбранного цвета.

2 Математические основы преобразования моделей

В приложении RGB используется как базовое представление цвета. Каналы RGB хранятся в виде целых чисел от 0 до 255. Перед вычислениями они нормализуются к диапазону $[0, 1]$:

$$r = \frac{R}{255}, \qquad g = \frac{G}{255}, \qquad b = \frac{B}{255}. \tag{1}$$

2.1 RGB $\Longleftrightarrow$ CMYK

RGB — аддитивная модель, описывающая смешение красного (Red), зелёного (Green) и синего (Blue) света. CMYK — субтрактивная модель, применяемая преимущественно при печати и использующая голубую (Cyan), пурпурную (Magenta), жёлтую (Yellow) и чёрную (Key) составляющие.

Для преобразования нормализованного RGB в CMYK сначала находится доля чёрного:

$$K = 1 - \max(r, g, b). \tag{2}$$

Если $K = 1$, цвет является чёрным и используется представление $C = M = Y = 0,\ K = 1$. В остальных случаях:

$$C = \frac{1 - r - K}{1 - K}, \qquad M = \frac{1 - g - K}{1 - K}, \qquad Y = \frac{1 - b - K}{1 - K}. \tag{3}$$

Обратное преобразование выполняется по формулам:

$$\begin{aligned} r &= (1 - C)(1 - K), \\ g &= (1 - M)(1 - K), \\ b &= (1 - Y)(1 - K). \end{aligned} \tag{4}$$

Значения C, M, Y и K в формулах задаются долями от 0 до 1. В интерфейсе они отображаются в процентах, поэтому перед вычислениями делятся на 100.

2.2 RGB $\Longleftrightarrow$ HSV

HSV описывает цвет при помощи тона (Hue), насыщенности (Saturation) и значения яркости (Value). Тон H задаётся в градусах, S и V — в диапазоне от 0 до 100%.

Обозначим максимальный и минимальный каналы:

$$M = \max(r, g, b), \qquad m = \min(r, g, b), \qquad \Delta = M - m. \tag{5}$$

Тогда насыщенность и значение яркости вычисляются так:

$$V = M, \qquad S = \begin{cases} 0, & M = 0, \\ \dfrac{\Delta}{M}, & M \neq 0. \end{cases} \tag{6}$$

Тон определяется в зависимости от максимального канала:

$$H = \begin{cases} 0, & \Delta = 0, \\ 60^\circ \left(\left(\dfrac{g-b}{\Delta} \right) \bmod 6 \right), & M = r, \\ 60^\circ \left(\dfrac{b-r}{\Delta} + 2 \right), & M = g, \\ 60^\circ \left(\dfrac{r-g}{\Delta} + 4 \right), & M = b. \end{cases} \tag{7}$$

Для преобразования HSV обратно в RGB используются нормализованные значения $H' = H/60$, S и V, где $S, V \in [0, 1]$:

$$C = V \cdot S, \qquad X = C\left(1 - \left|(H' \bmod 2) - 1\right|\right), \qquad m = V - C. \tag{8}$$

В зависимости от сектора тона H выбирается тройка (R', G', B'):

$$(R', G', B') = \begin{cases} (C, X, 0), & 0 \le H < 60, \\ (X, C, 0), & 60 \le H < 120, \\ (0, C, X), & 120 \le H < 180, \\ (0, X, C), & 180 \le H < 240, \\ (X, 0, C), & 240 \le H < 300, \\ (C, 0, X), & 300 \le H \le 360. \end{cases} \tag{9}$$

Затем $r = R' + m$, $g = G' + m$, $b = B' + m$, а для получения 8-битного RGB каждый канал умножается на 255 и округляется до целого числа.

2.3 Округление и особые случаи

В реализации RGB является базовым представлением цвета с 8 битами на канал. Поэтому при вводе значений CMYK или HSV приложение сначала преобразует их в RGB и округляет каналы, после чего пересчитывает CMYK и HSV из полученного RGB. Из-за такого округления исходные десятичные значения могут незначительно измениться. Для серого цвета тон HSV не определён математически; в приложении для него отображается $H = 0^\circ$.

3 Среда разработки и используемые средства

- **Язык программирования:** Python 3.
- **Графический интерфейс:** Tkinter и ttk.
- **Преобразование RGB/HSV:** стандартный модуль `colorsys`.
- **Проверка и обработка ввода:** стандартные модули `re` и `tkinter.messagebox`.
- **Тип приложения:** настольное приложение.

Сторонние Python-пакеты для работы приложения не требуются. Необходимо, чтобы установленная версия Python содержала Tkinter.

4 Интерфейс и реализация приложения

В главном окне отображаются образец выбранного цвета, его HEX-представление и три панели с компонентами RGB, CMYK и HSV. Для каждой координаты доступны поле точного ввода и ползунок. Диапазоны координат приведены в таблице 1.

Таблица 1: Диапазоны координат цветовых моделей

Модель	Компоненты	Диапазон
RGB	R, G, B	целые числа от 0 до 255
CMYK	C, M, Y, K	от 0 до 100%
HSV	H	от 0° до 360°
HSV	S, V	от 0 до 100%

Цвет также можно задать несколькими общими средствами: выбрать образец из быстрой палитры, открыть системный диалог выбора цвета или ввести HEX-код в формате `#RRGGBB`. После подтверждения поле ввода, ползунки и образец синхронизируются с новым цветом. Для возврата к начальному цвету предусмотрена кнопка сброса.

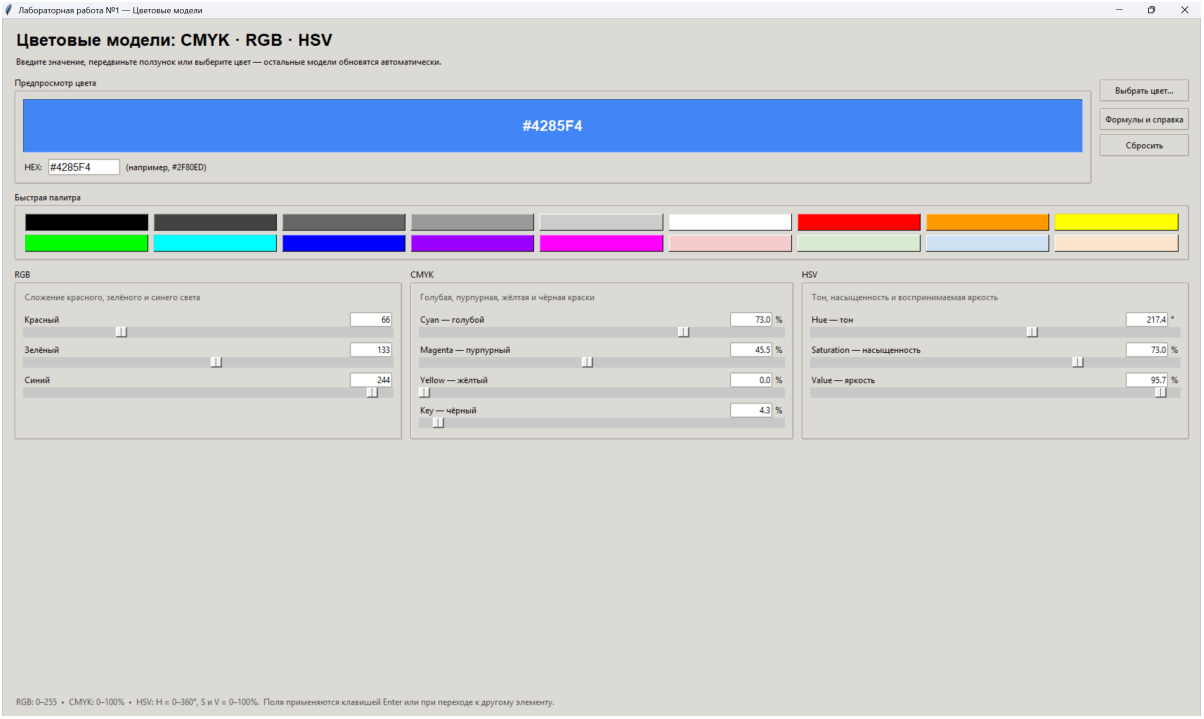


Рис. 1: Главное окно приложения для работы с цветовыми моделями.

Основная логика вынесена в класс `ColorModelsApp`. Метод `_apply_rgb()` сохраняет новый цвет в RGB и пересчитывает представления CMYK и HSV, обновляет текстовые поля, ползунки, HEX-код и цветовой образец. Метод `_commit_model()` обрабатывает введённые значения и проверяет их диапазоны. Метод `_on_slider()` обрабатывает перемещение ползунка. Метод `_commit_hex()` проверяет HEX-код, а `_choose_color()` открывает системный диалог выбора цвета.

Для синхронизации виджетов применяется флаг `_updating`: он предотвращает повторную обработку значений, которые меняются программно при пересчёте модели. При некорректном вводе приложение показывает сообщение об ошибке и возвращает отображение к последнему корректному цвету.

Основная последовательность работы программы:

1. Пользователь вводит координату, перемещает ползунок или выбирает цвет.
2. Введённые значения проверяются и переводятся в RGB.
3. RGB-каналы округляются и ограничиваются диапазоном от 0 до 255.
4. По актуальным RGB-каналам пересчитываются CMYK, HSV и HEX.
5. Интерфейс обновляет все поля, ползунки и предпросмотр цвета.

В методе обновления цвета реализован расчёт CMYK по нормализованным каналам RGB. Для чёрного цвета отдельно обрабатывается случай $K = 1$, чтобы избежать деления на ноль. HSV пересчитывается функцией стандартного модуля `colorsys`:

Листинг 1: Фрагмент метода обновления цветовых моделей

```
k = 1 - max(rn, gn, bn)
if k >= 1 - 1e-12:
    c = m = y = 0.0
    k = 1.0
else:
```

```
    c = (1 - rn - k) / (1 - k)
    m = (1 - gn - k) / (1 - k)
    y = (1 - bn - k) / (1 - k)

h, s, v = colorsys.rgb_to_hsv(rn, gn, bn)
```

5 Проверка работы приложения

Для проверки преобразований можно использовать основные цвета, представленные в таблице 2. Значения CMYK указаны в процентах; координаты HSV приведены как H в градусах, S и V в процентах.

Таблица 2: Контрольные значения для проверки моделей

Цвет	**RGB**	**CMYK, %**	**HSV**
Чёрный	$(0, 0, 0)$	$(0, 0, 0, 100)$	$(0°, 0, 0)$
Белый	$(255, 255, 255)$	$(0, 0, 0, 0)$	$(0°, 0, 100)$
Красный	$(255, 0, 0)$	$(0, 100, 100, 0)$	$(0°, 100, 100)$
Зелёный	$(0, 255, 0)$	$(100, 0, 100, 0)$	$(120°, 100, 100)$
Синий	$(0, 0, 255)$	$(100, 100, 0, 0)$	$(240°, 100, 100)$

Во время проверки следует изменить каждую координату RGB, CMYK и HSV и убедиться, что остальные представления обновляются. Дополнительно следует проверить выбор цвета из палитры, ввод HEX-кода, системный диалог, сброс, а также обработку значений за пределами допустимых диапазонов.

6 Выводы

В ходе лабораторной работы были рассмотрены цветовые модели RGB, CMYK и HSV и реализовано настольное приложение на Python для интерактивной работы с цветом.

В приложении реализованы точный ввод координат, управление компонентами с помощью ползунков, быстрая палитра, системный выбор цвета и ввод HEX-кода. При изменении цвета автоматически пересчитываются значения всех трёх моделей и обновляется визуальный образец. Добавлены проверки диапазонов и сообщение об ошибочном вводе.

Поскольку координаты RGB представлены 8-битными целыми числами, после преобразования CMYK или HSV возможны небольшие отклонения от введённых десятичных значений. Таким образом, приложение демонстрирует принципы преобразования и взаимосвязь цветовых моделей на практическом примере.

7 Источники

1. Python Software Foundation. Документация стандартного модуля `colorsys`. URL: `https://docs.python.org/3/library/colorsys.html`.
2. Python Software Foundation. Документация Tkinter. URL: `https://docs.python.org/3/library/tkinter.html`.